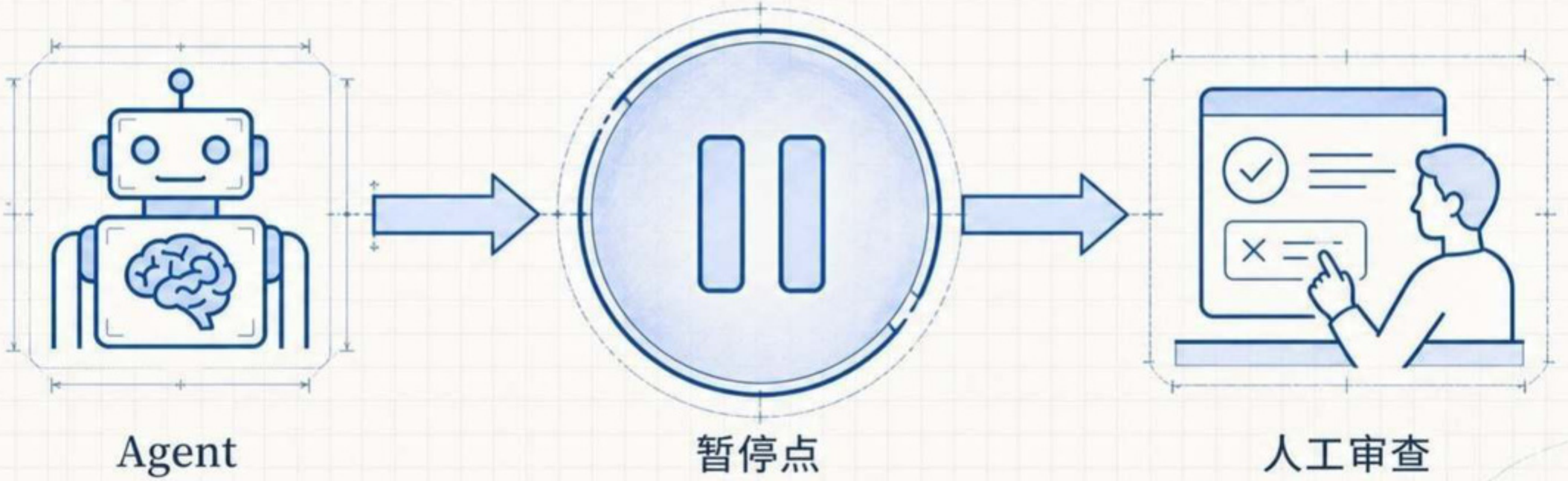
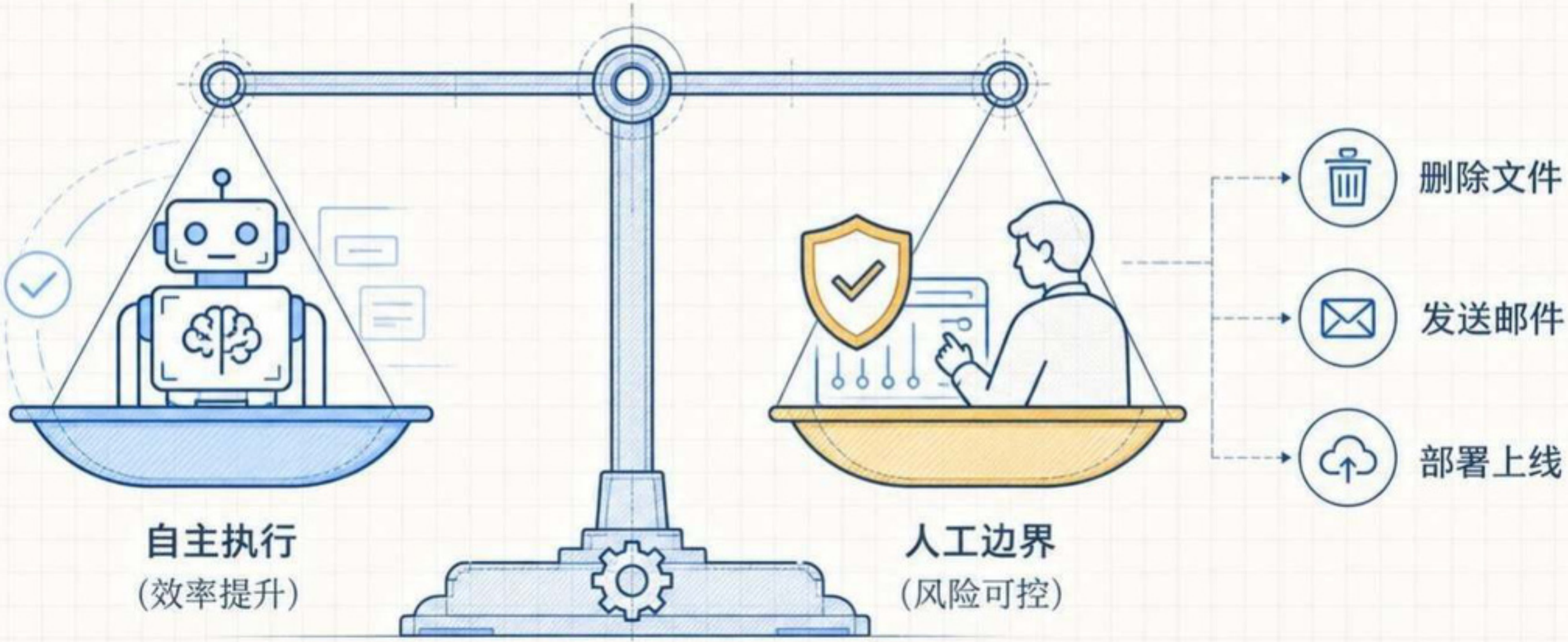
Deep Agents 实战

第 12 讲：HITL 基础 — 配置与决策



为什么需要 HITL？

自主性越高，越需要安全边界



工具审批的最小配置

```python
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

agent = create_deep_agent(
    model=model,
    tools=[send_email],
    interrupt_on={"send_email": {
      "allowed_decisions": ["approve", "edit", "reject"]
    }},
    checkpointer=MemorySaver(),
)
```

interrupt_on：
工具执行前人工审查

checkpointer：
保存可恢复的图状态

同一个 thread_id：
定位原来的暂停任务

⚠ MemorySaver 适合进程内演示；跨进程恢复需要持久化 Checkpointer。审查权限需由应用验证。

四种决策怎么配

不同工具开放不同人工动作

approve 通过

edit 修改

```
interrupt_on = {
    "delete_file": {"allowed_decisions": ["approve", "edit", "reject"]},
    "ask_user": {"allowed_decisions": ["respond"]},
    "read_file": False,
}
```

reject 拒绝

副作用工具：
只允许 approve / edit / reject

ask_user：
只开放 respond

只读工具：
无需中断

respond 不是拒绝
高风险用 reject

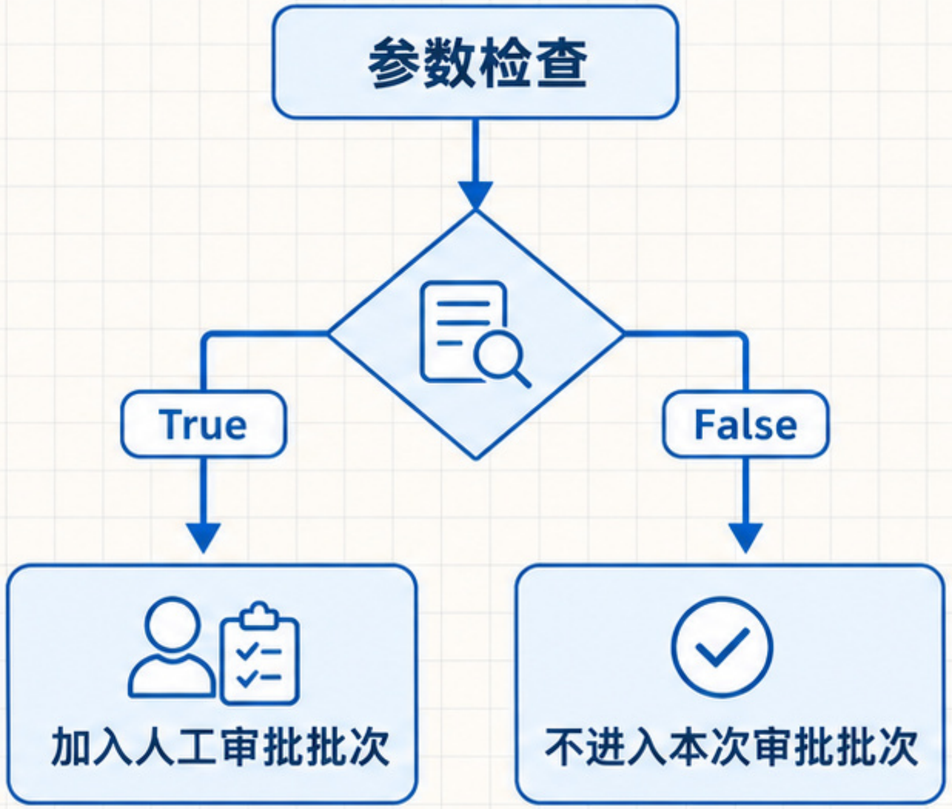
条件中断：按参数筛选

```python
def needs_review(request):
    return request.tool_call["args"].get("to") != "team@example.com"

interrupt_on = {"send_email": {
    "allowed_decisions": ["approve", "reject"],
    "when": needs_review,
}}
```

示例规则：仅团队地址自动放行；实际规则由应用按风险设计。

when 需要 langchain >= 1.3.3。字符串前缀判断不是文件系统隔离，路径安全需规范化与权限策略。



发起请求：带上 thread_id

恢复时必须回到同一条线程

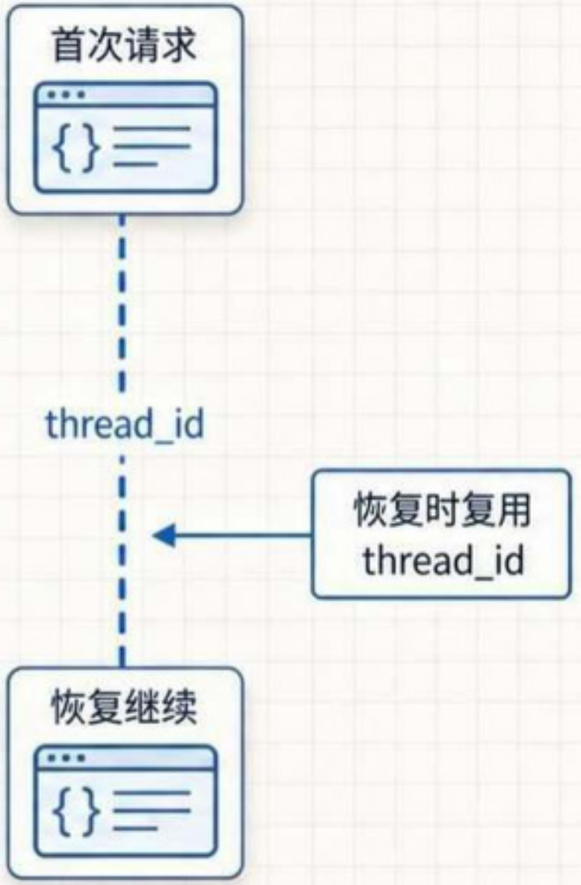
生成唯一 thread_id

```python
import uuid

config = {"configurable": {"thread_id": str(uuid.uuid4())}}

result = agent.invoke(
    {"messages": [{"role": "user", "content": "删除 temp.txt"}]},
    config=config,
    version="v2",
)
```

请求携带同一个 config



读取中断：展示原始请求

```python
value = result.interrupts[0].value

requests = value["action_requests"]

reviews = {r["action_name"]: r
           for r in value["review_configs"]}

for action in requests:
    args = action.get("arguments", action.get("args", {}))
    review = reviews[action["name"]]
    allowed = review["allowed_decisions"]
    print(action["name"], args, allowed)
```

读取中断值

提取请求列表

构建审核配置索引

遍历请求并关联审核

待用户审批

write_file

```
file_path:
 /reports/summary.md
content:
 ...
```

允许决策按 review_configs 显示。

批准　　拒绝

⚠ 这里采用 version="v2"；默认返回字典时读取 __interrupt__。保留原始参数，不要在展示中改写。

approve：批准请求，不保证成功

```python
from langgraph.types import Command

decisions = [{"type": "approve"}]
command = Command(resume={
    "decisions": decisions
})
result = agent.invoke(
    command,
    config=config,
    version="v2",
)
```



原始请求
write_file

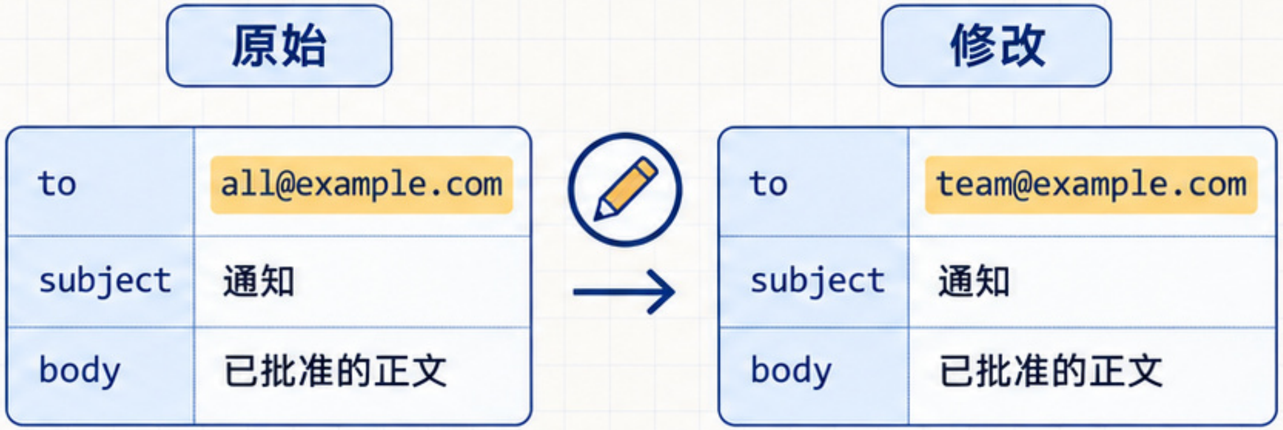
```
{
  "file_path":
   "/reports/summary.md",
  "content": "..."
}
```

> **approve 不是成功凭证。权限校验、工具错误或后续步骤仍可能失败；外部副作用应独立核验。**

edit：保留完整参数，只改必要项

PYTHON

```python
original = action.get("arguments", action.get("args", {}))
new_args = {**original, "to": "team@example.com"}
decisions = [{
    "type": "edit",
    "edited_action": {
        "name": action["name"],
        "args": new_args,
    },
}]
```



edited_action 必须包含 name 与 args。

subject、body 等必需参数不可遗漏。

提交前重新展示完整动作。

resume： reject

拒绝工具执行，并把原因反馈给 Agent

PYTHON

```
decisions = [{
    "type": "reject",
    "message": "用户拒绝删除。不要重试，
               请询问是否改为归档。"
}]

agent.invoke(
    Command(resume={"decisions": decisions}),
    config=config,
    version="v2",
)
```

拟执行工具



delete_file

args: {"path": "report.pdf"}

反馈给 Agent

用户拒绝删除。
不要重试，请询问是否改为归档。

1 工具不会执行

2 message 会返回给 Agent

3 用于删除、发送、部署等高风险动作

拒绝副作用工具时用 reject，不要用 respond

用户拒绝 → 返回反馈 → Agent 重新规划

respond：人类提供工具结果

PYTHON

```python
decisions = [{
  "type": "respond",
  "message": "使用季度维度，并排除测试数据。"
}]
```



1. 工具函数不会执行
2. 人的 message 成为 content
3. ToolMessage.status = "success"

不同意删除、发送或部署时用 reject。respond 不是软拒绝，也不是操作已发生的证据。

批量工具调用：按顺序恢复

decisions 必须和 action_requests 一一对应

action_requests

1. delete_file(
   path="temp.txt")
2. send_email(
   to="admin@example.com")
3. ask_user(
   question="使用哪个周期？")

1 → 1

2 → 2

3 → 3

PYTHON

```python
decisions = [
    {"type": "approve"},
    {"type": "reject", "message": "不要发送邮件。"},
    {"type": "respond", "message": "使用季度。"},
]

agent.invoke(
    Command(resume={"decisions": decisions}),
    config=config,
    version="v2"
)
```

数量相同 | 顺序相同 | 每个动作一个 decision

子 Agent：覆盖审批策略

PYTHON

```python
subagents = [{
    "name": "file-manager",
    "description": "审查并管理文件",
    "tools": [inspect_file, remove_file],
    "interrupt_on": {
      "inspect_file": {"allowed_decisions": ["approve", "reject"]},
      "remove_file": {"allowed_decisions": ["approve", "reject"]},
    },
}]
```

父 Agent 策略

未设置时继承

子 Agent interrupt_on

显式覆盖

inspect_file / remove_file 为应用已定义的工具。
覆盖时完整列出需要保护的工具。

审批策略不等于独立文件系统隔离。

路径审批：覆盖与删除也要审查

```python
from deepagents import FilesystemPermission, create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

agent = create_deep_agent(
    model=model,
    permissions=[FilesystemPermission(
        operations=["write"],
        paths=["/secrets/**"],
        mode="interrupt",
    )],
    checkpointer=MemorySaver(),
)
```

- 内置文件工具按路径检查
- write 涵盖写入、覆盖、编辑和删除
- 受保护后代可阻止整次目录删除

deepagents >= 0.6.8；v0.7 支持覆盖与递归删除。

不是任意 Python/自定义工具/Shell 的安全沙箱；执行型后端的配置限制见第11章。

本讲回顾

把工具审批变成可恢复的 Agent 流程

- interrupt_on 决定哪些工具需要人工审查
- allowed_decisions 限定 approve / edit / reject / respond
- thread_id + checkpointer 保存暂停现场
- action_requests 与 decisions 一一对应
- 子 Agent 可以按职责单独配置中断



人类不是旁路，而是运行时的一部分

下一讲： HITL 进阶 —— interrupt()、 checkpoint 与恢复规则